Lecture 8 & 9

STL Containers

STL Containers

STL - Standard Template Library
STL consists of 4 key parts - algorithms, containers, functions, iterators

- Containers - store data
- Algorithms - perform operations on data
- Iterators - provide interfaces for containers, allowing to use them in a simple way
  - Also connect algorithms with containers
- Functions - modify the behaviour of algorithms (when passed as a function object)

STL containers, ejudge reference

STL Containers

STL - Standard Template Library
STL provides us with various kinds of containers.

There are three classes of containers - sequence containers, associative containers, and unordered associative containers - each of which is designed to support a different set of operations.

STL containers, ejudge reference

Vector

`vector` is an indexed sequence container that encapsulates dynamic size arrays

- requires `<vector>` header (`#include <vector>`)
- dynamic in size
- contiguous in memory

```
vector<int> v = {1, 2, 3, 4, 5};
for(int i = 0; i < v.size(); i++)
    cout << v[i] << endl;
```

vector, ejudge reference

Pair

`pair` provides a way to store two heterogeneous (different) objects as a single unit

- to create a pair, use `make_pair(val1, val2)` function
- to access the first or the second value in a pair, use `pair.first` and `pair.second`

```
pair<int, bool> p1 = make_pair(1, true);
pair<string, double> p2 = make_pair("Value of Pi:", 3.14);
cout << p1.first << " " << p1.second << endl;
cout << p2.first << " " << p2.second << endl;
```

pair, ejudge reference || pair, cplusplus.com

Map

`map` is an associative container that stores key-value pairs with unique keys, which are following a specific order

- requires `<map>` header (`#include <map>`)
- no duplicate keys allowed
- always sorted
- the value of an element is identified by its key

```
map<string, int> m;
for(int i = 1; i <= 5; i++) {
    string s; cin >> s;
    m[s] = i;
}
map<string, int>::iterator it;
for(it = m.begin(); it != m.end(); it++) cout << it->first << " " << it->second << endl;
```

map, ejudge reference || map, cplusplus.com

Iterators

Iterators are objects that point to elements within a container (arrays, vectors, lists, sets, maps, etc.) in the C++ STL. They allow you to traverse and manipulate the elements in these containers without needing to know the underlying structure of the container.

Types of iterators:

- Input Iterators
- Output Iterators
- Forward Iterators
- Bidirectional Iterators
- Random Access Iterators

Operations, supported by iterators

Iterator operation	Description
All iterators	
`++p`	Preincrement an iterator.
`p++`	Postincrement an iterator.
Input iterators	
`*p`	Dereference an iterator.
`p = p1`	Assign one iterator to another.
`p == p1`	Compare iterators for equality.
`p != p1`	Compare iterators for inequality.
Output iterators	
`*p`	Dereference an iterator.
`p = p1`	Assign one iterator to another.
Forward iterators	Forward iterators provide all the functionality of both input iterators and output iterators.

Iterator operation	Description
Bidirectional iterators	
`--p`	Predecrement an iterator.
`p--`	Postdecrement an iterator.
Random-access iterators	
`p += i`	Increment the iterator `p` by `i` positions.
`p -= i`	Decrement the iterator `p` by `i` positions.
`p + i` *or* `i + p`	Expression value is an iterator positioned at `p` incremented by `i` positions.
`p - i`	Expression value is an iterator positioned at `p` decremented by `i` positions.
`p - p1`	Expression value is an integer representing the distance between two elements in the same container.
`p[ i ]`	Return a reference to the element offset from `p` by `i` positions
`p < p1`	Return `true` if iterator `p` is less than iterator `p1` (i.e., iterator `p` is *before* iterator `p1` in the container); otherwise, return `false`.
`p <= p1`	Return `true` if iterator `p` is less than or equal to iterator `p1` (i.e., iterator `p` is *before* iterator `p1` or *at the same location* as iterator `p1` in the container); otherwise, return `false`.
`p > p1`	Return `true` if iterator `p` is greater than iterator `p1` (i.e., iterator `p` is *after* iterator `p1` in the container); otherwise, return `false`.
`p >= p1`	Return `true` if iterator `p` is greater than or equal to iterator `p1` (i.e., iterator `p` is *after* iterator `p1` or *at the same location* as iterator `p1` in the container); otherwise, return `false`.

Set

`set` is an associative container that stores unique elements following a specific order

- requires `<set>` header (`#include <set>`)
- no duplicates allowed
- always sorted
- elements are constant, i.e. cannot be changed
- the value of an element also identifies it

```
set<int> s;
for(int i = 0; i < 10; i++) s.insert(i);
set<int>::iterator it;
for(it = s.begin(); it != s.end(); it++)
        cout << *it << " ";
```

set, ejudge reference || set, cplusplus.com

Stack

`stack` is a container adapter that works on the FILO (First In Last Out) principle

- requires `<stack>` header (`#include <stack>`)
- the first value added to the stack will be accessed the last
- supports only `push()` to add elements to the top and `pop()` to remove them, also from the top
- access is only possible to the top element, using `top()`

```
stack<int> s;
for(int i = 1; i <= 5; i++) s.push(i);
while(!s.empty()) {
    cout << s.top() << endl;
    s.pop();
}
```

stack, ejudge reference

Queue

`queue` is a container adapter that works on the FIFO (First In First Out) principle

- requires `<queue>` header (`#include <queue>`)
- the first value added to the queue will be accessed the first
- supports only `push()` to add elements to the back and `pop()` to remove them from the front
- access only to the first element with `front()` and last element with `back()`

```
queue<int> q;
for(int i = 1; i <= 5; ++i) q.push(i);
while(!q.empty()) {
    cout << q.front() << endl;
    q.pop();
}
```

queue, ejudge reference

Deque

`deque` is an indexed sequence container allows quick insertions both to the beginning and end

- requires `<deque>` header (`#include <deque>`)
- dynamic in size
- not contiguous in memory, opposed to vector
- allows quick insertion not only at the end, but also at the start of the container

```
deque<int> dq;
for(int i = 1; i <= 5; ++i) dq.push_back(i);
for(int i = 0; i < dq.size(); ++i) cout << dq[i] << " ";
cout << endl;
```

deque, ejudge reference